

The Better Way to Run GitHub Actions

Self-Hosted Runners on AWS ECS

Most organizations start with an EC2 self-hosted runner. It's simple, but there's a catch: that EC2 instance runs continuously, burning money even when no workflows are executing. A more scalable, production-ready approach? Running your GitHub Runner inside an Amazon ECS Task.

GitHub-Hosted vs. Self-Hosted Runners

While GitHub-hosted runners are convenient to start with, many enterprise teams quickly transition to self-hosted runners. Here is why self-hosted runners are preferred for production workloads:

-  **Network Security & VPC Access:** Self-hosted runners live inside your own AWS VPC. This means they can securely access internal databases, private APIs, and internal services without exposing them to the internet.
-  **Custom Hardware & Tooling:** You control the vCPU, memory, storage, and the base image. You can pre-install heavy dependencies to drastically speed up builds or use specialized hardware (like GPUs).
-  **Cost Efficiency:** For organizations with heavy CI/CD pipelines, paying for GitHub's per-minute compute can become very expensive. Running your own runners (especially on ECS Fargate) cuts down the bill.
-  **No Artificial Limits:** You are not restricted by GitHub's default job timeouts or standard compute limits, giving you full control over long-running jobs.

Why ECS Over EC2?

Even among self-hosted options, running your runners on ECS (specifically using AWS Fargate) provides massive benefits over static EC2 instances:

-  **Lower Infrastructure Cost:** Only pay for the compute you use during builds instead of paying for a 24/7 idle EC2 instance.
-  **Better Scalability:** Easily scale the number of runners up or down based on queue depth.
-  **Fresh Environments:** Ephemeral runners mean every build starts clean, avoiding "it works on my machine" runner drift and leftover state.

- **✓ Easier Upgrades:** Manage your runner environment entirely via Docker images.
- **✓ Better Security & Isolation:** Leverage ECS Task Roles for fine-grained IAM permissions instead of managing long-lived access keys.

The ECS Runner Architecture

Here is how the flow looks in production:

- 1. Trigger:** Developer pushes code or opens a PR.
- 2. GitHub Actions:** The workflow triggers, looking for a runner with `runs-on: self-hosted`.
- 3. ECS Task Starts:** An ECS container (the GitHub Runner) spins up.
- 4. Execution:** The runner checks out code, sets up Python, runs tests, and builds the package.
- 5. Artifacts:** Build artifacts are uploaded (e.g., to Amazon S3).
- 6. Task Stops:** The ephemeral runner container shuts down.

Step-by-Step Implementation Guide

1. Register the Runner in GitHub

- 1. Go to Repository Settings:** Navigate to your repository (or organization) settings in GitHub.
- 2. Navigate to Actions:** Click on **Actions** > **Runners** in the left sidebar.
- 3. Add New Runner:** Click the **New self-hosted runner** button.
- 4. Select OS and Architecture:** Choose **Linux** and the appropriate architecture (e.g., x64 or ARM64) matching your ECS tasks.
- 5. Copy the Token:** GitHub will display a configuration script. Look for the `config.sh` command and copy the `--token <YOUR_TOKEN>` value. You will need this for your ECS task environment variables.

2. Build the GitHub Runner Docker Image

Create a Dockerfile that includes the runner binaries and your required build tools (like Python, Git, etc.).

```
FROM ubuntu:24.04

# Install necessary build tools and dependencies
RUN apt-get update && apt-get install -y      curl      git      python3      python3-pip
python3-venv

RUN pip3 install pytest build

WORKDIR /actions-runner

# Download the GitHub Runner binaries (replace with the latest URL)
RUN curl -o actions-runner-linux-x64.tar.gz -L https://github.com/actions/runner/
releases/download/vX.Y.Z/actions-runner-linux-x64-X.Y.Z.tar.gz
RUN tar xzf ./actions-runner-linux-x64.tar.gz

# The entrypoint script will use the token to register and start the runner
# ENTRYPOINT ["/config.sh", "--unattended", "--url", "https://github.com/<org>/<repo>",
"--token", "${RUNNER_TOKEN}", "&&", "/run.sh"]
```

3. Push the Image to Amazon ECR

1. **Create ECR Repository:** Create a private repository in AWS ECR (e.g., `github-runner`).
2. **Build the Image:** Run `docker build -t github-runner .`
3. **Tag the Image:** Tag it for your ECR repo: `docker tag github-runner:latest <aws_account_id>.dkr.ecr.<region>.amazonaws.com/github-runner:latest`
4. **Push to ECR:** Push the image: `docker push <aws_account_id>.dkr.ecr.<region>.amazonaws.com/github-runner:latest`

4. Create the ECS Cluster

1. **Open ECS:** Navigate to Amazon ECS in the AWS Console.
2. **Create Cluster:** Click **Clusters** > **Create Cluster**.
3. **Name It:** Name it `github-runner-cluster` .
4. **Select Launch Type:** Choose **AWS Fargate** (recommended for production).

5. Create the ECS Task Definition

This definition tells ECS how to run your container and where to pull the image from.

- **Image URI:** `<aws_account_id>.dkr.ecr.<region>.amazonaws.com/github-runner:latest`
- **Compute:** 2 vCPU, 4 GB Memory.
- **Environment Variables:**
 - `RUNNER_NAME=ecs-runner`

- `RUNNER_LABELS=ecs,python,linux`
- `GITHUB_URL=https://github.com/<org>/<repo>`
- `RUNNER_TOKEN=<your-registration-token>`

6. Start the ECS Service

Create an ECS Service within your new cluster, pointing to your Task Definition, and set the **Desired Tasks** to `1` (or more). Once running, check GitHub > Settings > Actions > Runners. You should see `ecs-runner` listed with an **Idle** status.

Example Workflow

Target your new ECS runner in your workflow file (`.github/workflows/python-ci.yml`).

```
name: Python CI on ECS Self Hosted Runner

on:
  workflow_dispatch:
  push:
    branches:
      - main

jobs:
  build-test:
    # Target specific runner using custom labels
    runs-on:
      - self-hosted
      - ecs
      - python

    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v6

      - name: Setup Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.12"
          cache: pip

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install build pytest

      - name: Execute Unit Tests
        run: |
```

```
pytest -v

- name: Build Python Package
  run: |
    python -m build

- name: Upload Build Artifacts
  uses: actions/upload-artifact@v4
  with:
    name: python-package
    path: |
      dist/
```



Production Best Practices

1. Use Custom Labels: Don't just use `runs-on: self-hosted`. Use specific tags like `ecs`, `python`, `linux`.

2. Use Ephemeral Runners: Configure your runners to execute one job and die (`--ephemeral` flag during configuration). This ensures a clean environment every time.

3. Secure Secrets via IAM: Never hardcode credentials. When running on ECS, attach an **IAM Task Role** to the container task.

4. Cache Dependencies: Use native GitHub actions caching (like `cache: pip`) to significantly speed up repeated builds.

5. Publish Artifacts: Don't just run tests—upload the final build output so it can be consumed by downstream deployment stages.

This pattern combines the customizability of self-hosted runners with the operational simplicity of AWS Fargate container orchestration.